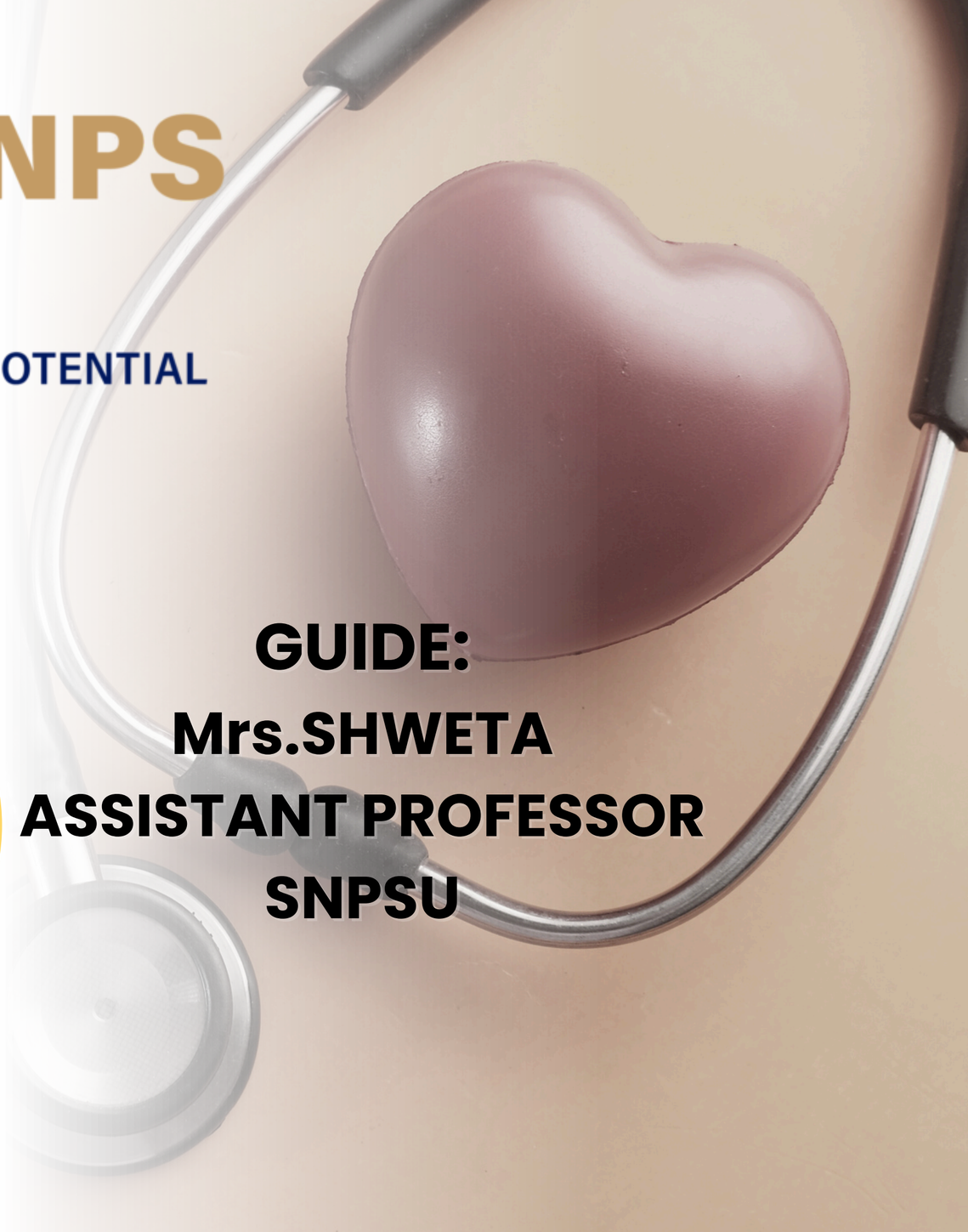
SAPTHAGIRI NPS UNIVERSITY

UNMATCHED EXCELLENCE,UNLIMITED POTENTIAL



EC-27

PRESENTED BY:

RAKSHA KALLUR 24SUUBECS1664
RAMYA K S 24SUUBECS1685
RAKSHITHA K 24SUUBECS1673
RANJITA G N 24SUUBECS1694
RANJITA N N 24SUUBECS1695

GUIDE:

Mrs.SHWETA
ASSISTANT PROFESSOR
SNPSU

PREDICTING HEART DISEASE

OBJECTIVE

INTRODUCTION

METHODOLOGY

SCREENSHOTS AND RESULTS

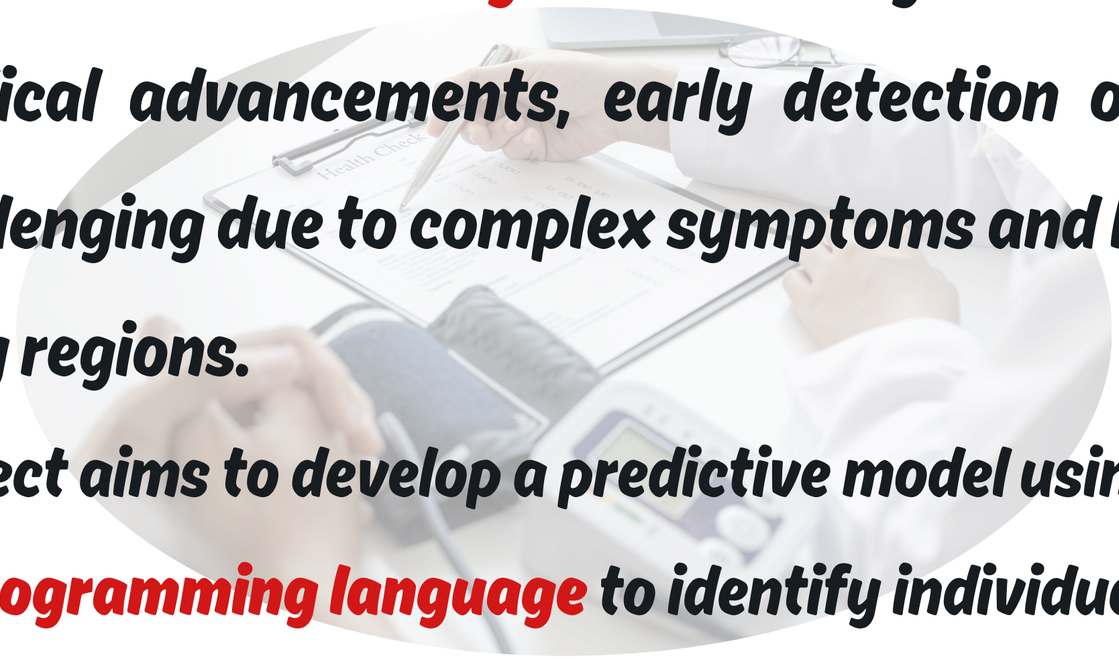
OBJECTIVES

- To design a **machine learning model** that can predict the risk of heart disease based on patient health parameters like age, cholesterol, blood pressure, etc.
- To use the **Python programming language** for implementing the entire workflow — from data preprocessing to model evaluation.
- To work with libraries such as: **pandas**, **numpy** , **scikit** ,**Streamlit**,
- To utilize the UCI Heart Disease Dataset as a reliable source of real-world data.
- To train, test, and compare classification algorithms like **Logistic Regression, Decision Tree, and Random Forest.**
- To develop a user-friendly web app where users can enter medical details and get instant predictions.
- To support early diagnosis and preventive healthcare using data-driven technology.

INTRODUCTION

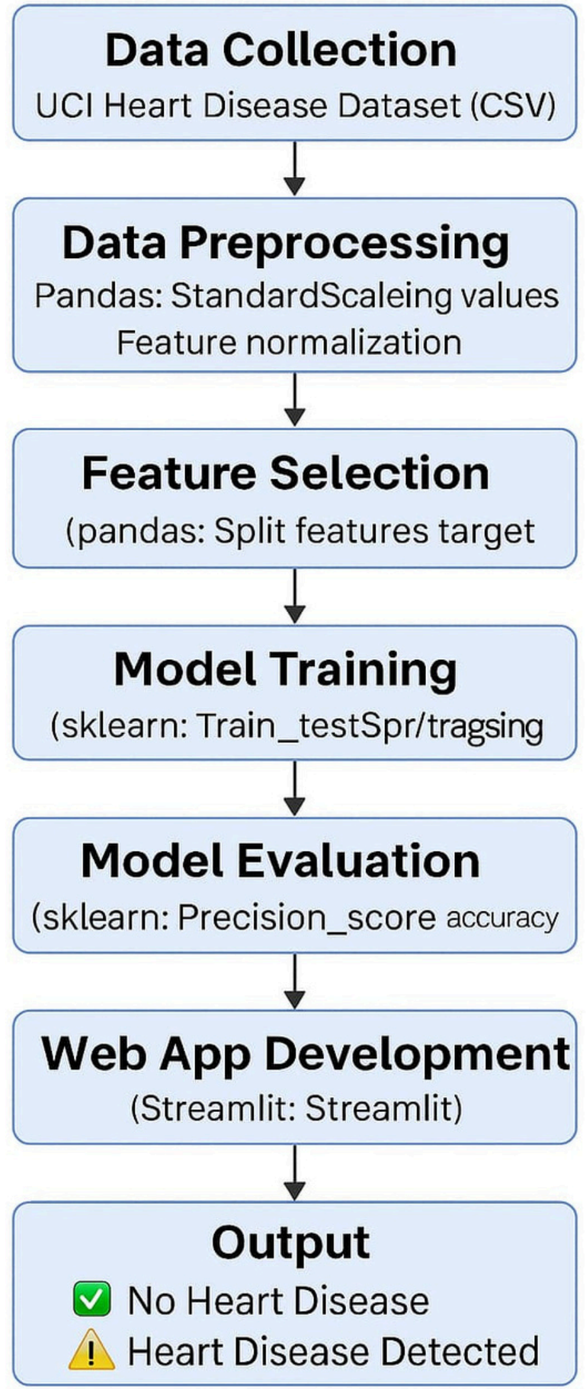
Heart disease remains the leading cause of death worldwide, accounting for over **17.9 million deaths each year** according to the World Health Organization. Despite medical advancements, early detection of heart-related conditions remains challenging due to complex symptoms and limited access to diagnostic tools in many regions.

This mini project aims to develop a predictive model using **Machine Learning** (ML) and the **Python programming language** to identify individuals at risk of heart disease. By analyzing patient data such as age, cholesterol, blood pressure, and other clinical factors, the model can assist in early diagnosis and support timely medical intervention.



- Utilizes **Python** for developing the entire prediction pipeline.
- Implements Machine Learning algorithms to detect heart disease risk.
- Employs essential **Python libraries:**
- **pandas** for data handling
- **scikit-learn** for ML modeling
- **Streamlit** for building a user-friendly web interface
- Uses the **UCI Heart Disease Dataset** for training and testing.
- Aims to support early diagnosis through data-driven predictions.
- Delivers a simple interactive app where users **input health data** and **receive instant results.**

FLOWCHART

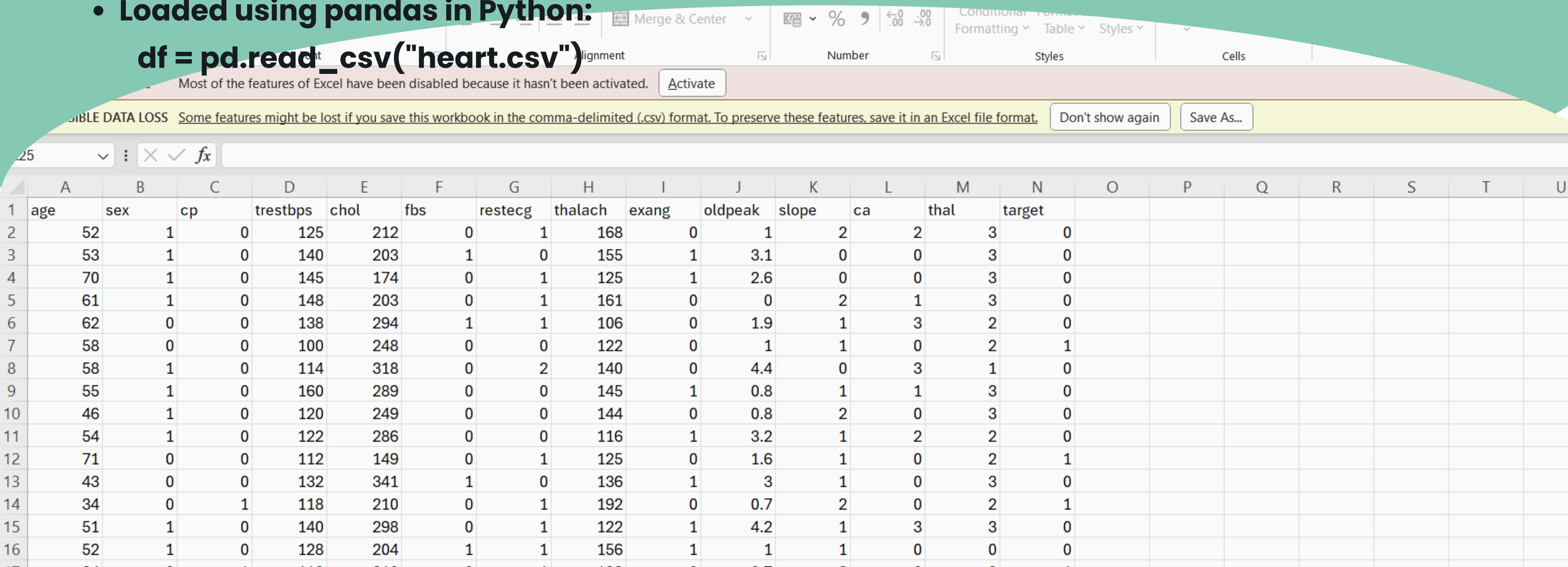

Data Collection
UCI Heart Disease Dataset (CSV)
Data Preprocessing
Pandas: StandardScaleing values
Feature normalization
Feature Selection
(pandas: Split features target
Model Training
(sklearn: Train_testSpr/tragsing
Model Evaluation
(sklearn: Precision_score accuracy
Web App Development
(Streamlit: Streamlit)
Output
✅ No Heart Disease
⚠️ Heart Disease Detected

This project follows a structured machine learning pipeline, fully implemented in the Python programming language using various Python libraries.

1. Data Collection

- **Dataset: UCI Heart Disease Dataset (CSV format)**
- **Loaded using pandas in Python:**

```
df = pd.read_csv("heart.csv")
```



	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
2	52	1	0	125	212	0	1	168	0	1	2	2	3	0
3	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
4	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
5	61	1	0	148	203	0	1	161	0	0	2	1	3	0
6	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0
7	58	0	0	100	248	0	0	122	0	1	1	0	2	1
8	58	1	0	114	318	0	2	140	0	4.4	0	3	1	0
9	55	1	0	160	289	0	0	145	1	0.8	1	1	3	0
10	46	1	0	120	249	0	0	144	0	0.8	2	0	3	0
11	54	1	0	122	286	0	0	116	1	3.2	1	2	2	0
12	71	0	0	112	149	0	1	125	0	1.6	1	0	2	1
13	43	0	0	132	341	1	0	136	1	3	1	0	3	0
14	34	0	1	118	210	0	1	192	0	0.7	2	0	2	1
15	51	1	0	140	298	0	1	122	1	4.2	1	3	3	0
16	52	1	0	128	204	1	1	156	1	1	1	0	0	0

2. **Data Preprocessing (Python-based)**

- Handled using Python libraries:
  - pandas for handling missing data and data manipulation.
  - StandardScaler from sklearn.preprocessing for feature normalization:
  - from sklearn.preprocessing import StandardScaler
  - scaler = StandardScaler()
  - X_scaled = scaler.fit_transform(X)

3. Feature Selection

- Features and target split in Python:
- X = df.drop("target", axis=1)
- y = df["target"]
- Data is scaled and cleaned using Python code.

4. Train-Test Split

- Performed using train_test_split from sklearn.model_selection:

  from sklearn.model_selection import train_test_split

  X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2)

5. Model Training

- Built using LogisticRegression from scikit-learn:

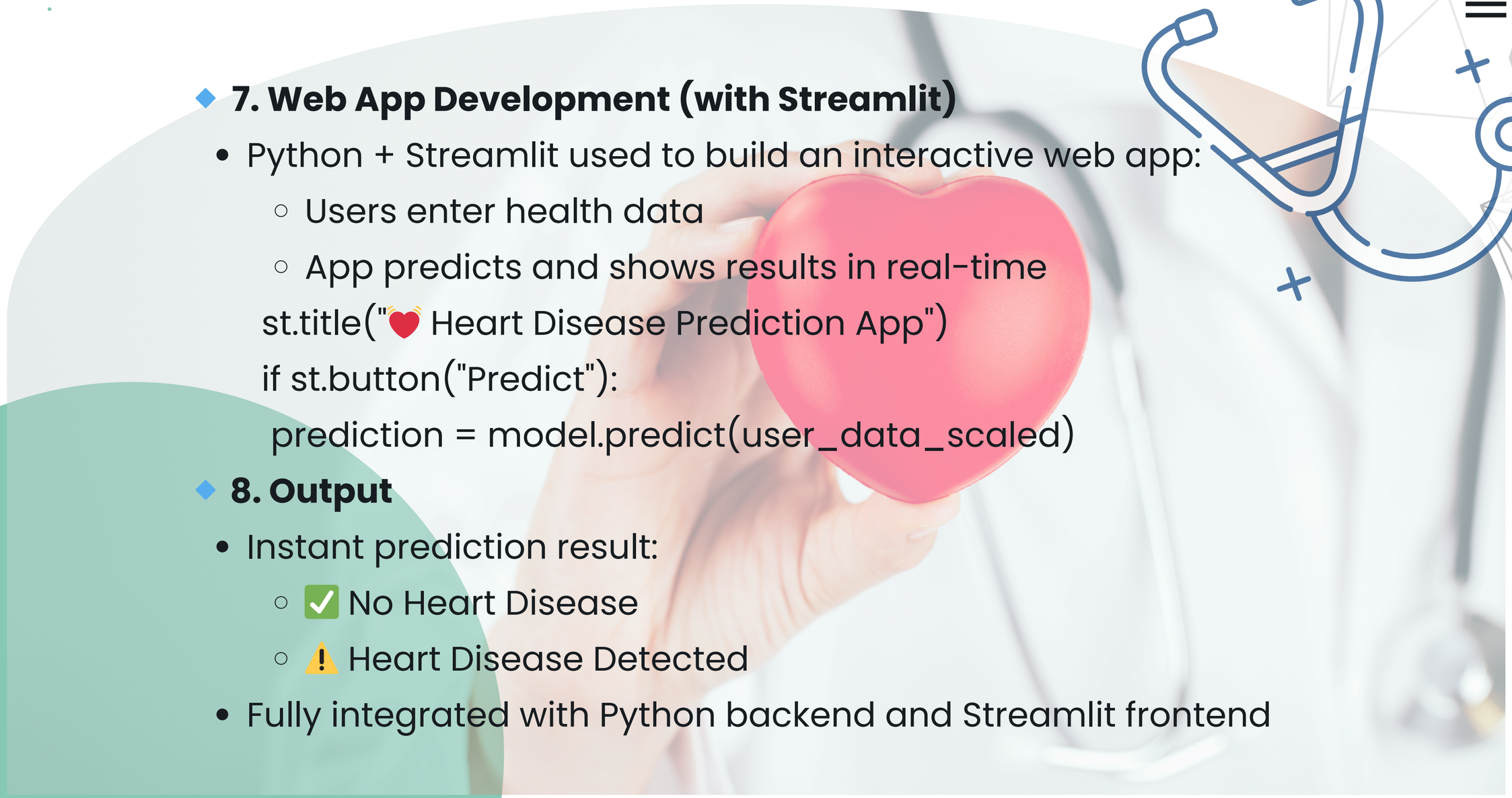
  from sklearn.linear_model import LogisticRegression

  model = LogisticRegression()

  model.fit(X_train, y_train)

6. Model Evaluation

- Evaluated using Python functions:
  - accuracy_score, precision_score, recall_score, f1_score
- Libraries used: sklearn.metrics



🔹 7. Web App Development (with Streamlit)

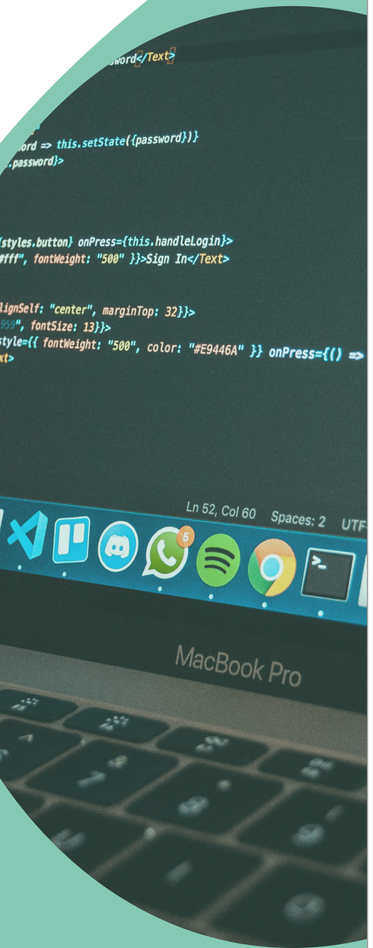
- Python + Streamlit used to build an interactive web app:
  - Users enter health data
  - App predicts and shows results in real-time

```
st.title("❤️ Heart Disease Prediction App")
if st.button("Predict"):
 prediction = model.predict(user_data_scaled)
```

🔹 8. Output

- Instant prediction result:
  - ✅ No Heart Disease
  - ⚠️ Heart Disease Detected
- Fully integrated with Python backend and Streamlit frontend

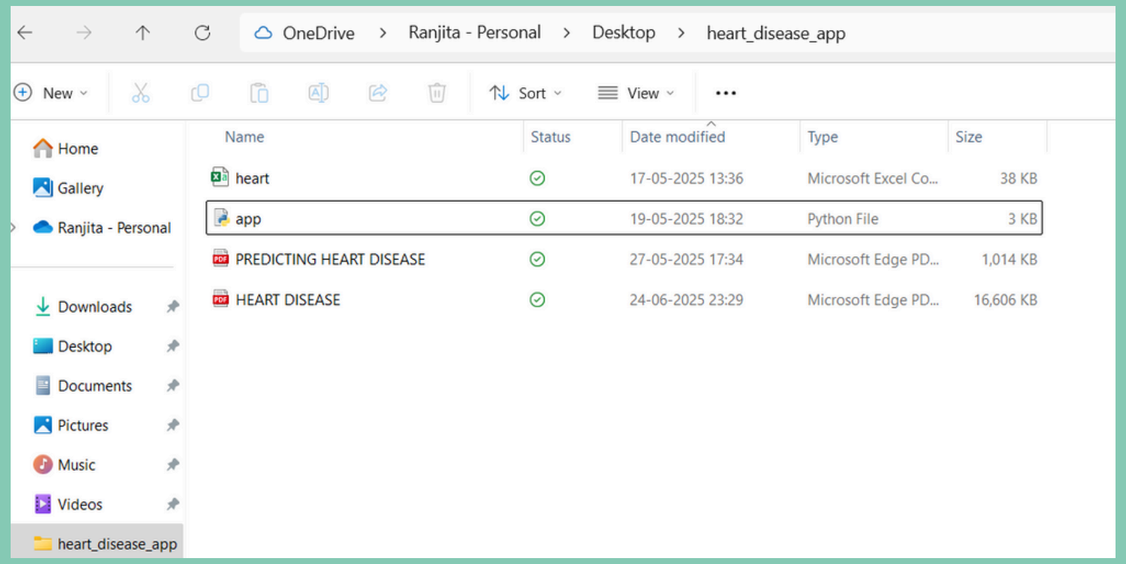
CODE

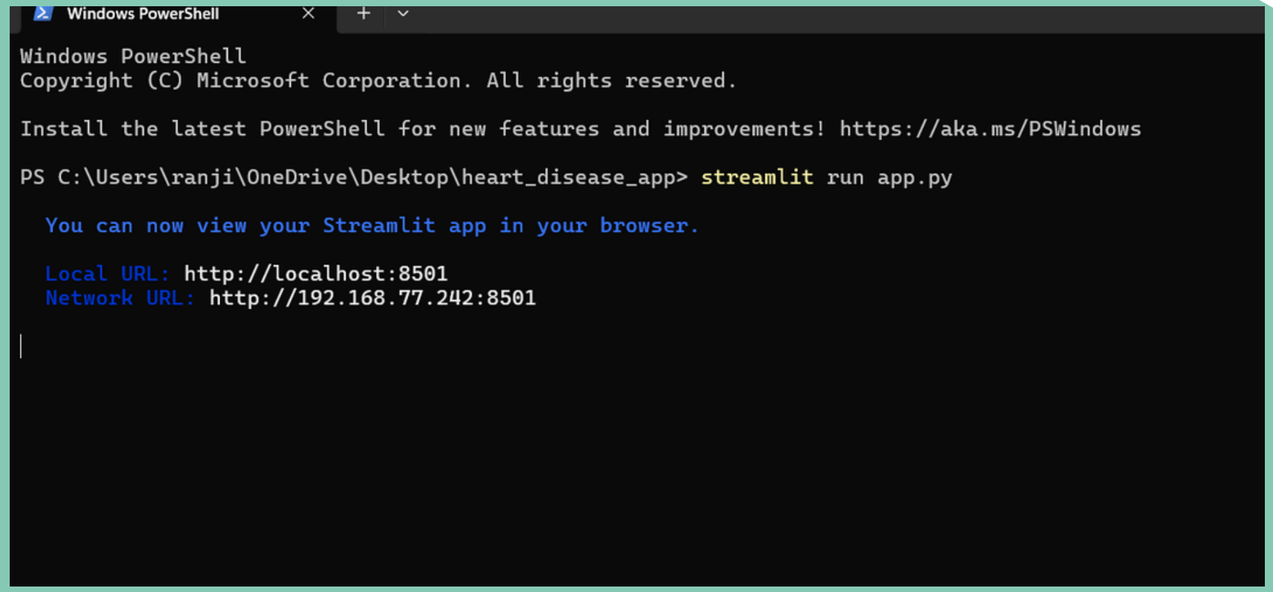



```python
# heart_disease_app.py
import streamlit as st
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
# Load dataset
@st.cache_data
def load_data():
    df = pd.read_csv("heart.csv")
    return df
df = load_data()
# Train model
X = df.drop("target", axis=1)
y = df["target"]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
model = LogisticRegression()
model.fit(X_train, y_train)
# Streamlit App
st.title("❤️ Heart Disease Prediction App")
st.write("Enter the following details:")
# User inputs
age = st.number_input("Age", min_value=1, max_value=120, value=45)
sex = st.selectbox("Sex", ["Male", "Female"])
cp = st.selectbox("Chest Pain Type (cp)", [0, 1, 2, 3])
```

```python
cp = st.selectbox("Chest Pain Type (cp)", [0, 1, 2, 3])
trestbps = st.number_input("Resting Blood Pressure (trestbps)", value=120)
chol = st.number_input("Serum Cholesterol in mg/dl (chol)", value=200)
fbs = st.selectbox("Fasting Blood Sugar > 120 mg/dl (fbs)", [0, 1])
restecg = st.selectbox("Resting ECG Results (restecg)", [0, 1, 2])
thalach = st.number_input("Max Heart Rate Achieved (thalach)", value=150)
exang = st.selectbox("Exercise Induced Angina (exang)", [0, 1])
oldpeak = st.number_input("Oldpeak (ST depression)", value=1.0)
slope = st.selectbox("Slope of ST segment (slope)", [0, 1, 2])
ca = st.selectbox("Number of major vessels (ca)", [0, 1, 2, 3])
thal = st.selectbox("Thalassemia (thal)", [0, 1, 2, 3])
# Prepare input for model
user_data = pd.DataFrame([[age, 1 if sex == "Male" else 0, cp, trestbps, chol,
                           fbs, restecg, thalach, exang, oldpeak, slope, ca, thal]],
                         columns=X.columns)
user_data_scaled = scaler.transform(user_data)
# Predict
if st.button("Predict"):
    prediction = model.predict(user_data_scaled)[0]
    if prediction == 1:
        st.error("⚠️ The model predicts **Heart Disease**.")
    else:
        st.success("✅ The model predicts **No Heart Disease**.")
st.markdown("---")
st.markdown("Made with ❤️ using Streamlit and Scikit-learn")
```

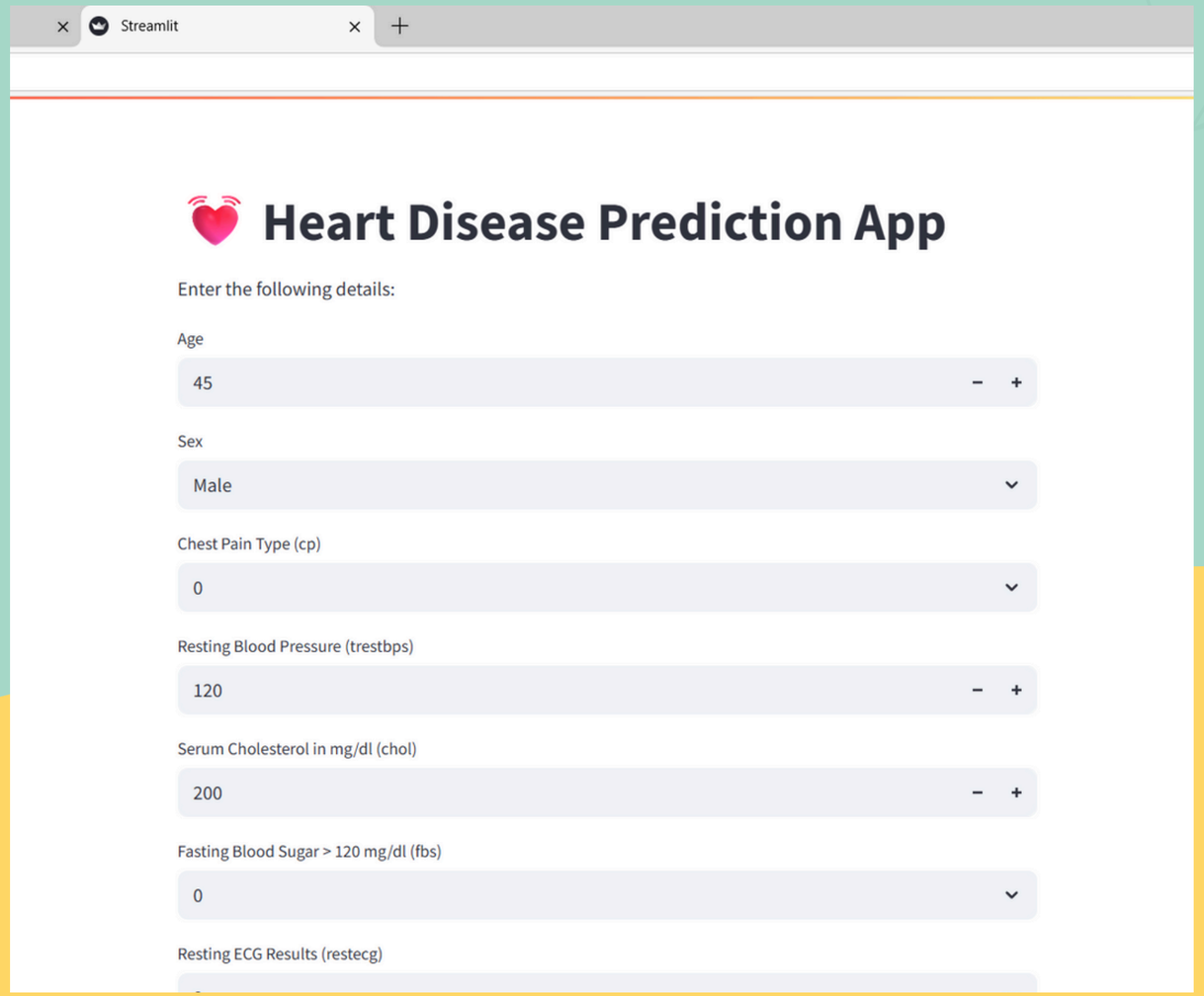
SCREENSHOTS AND RESULTS

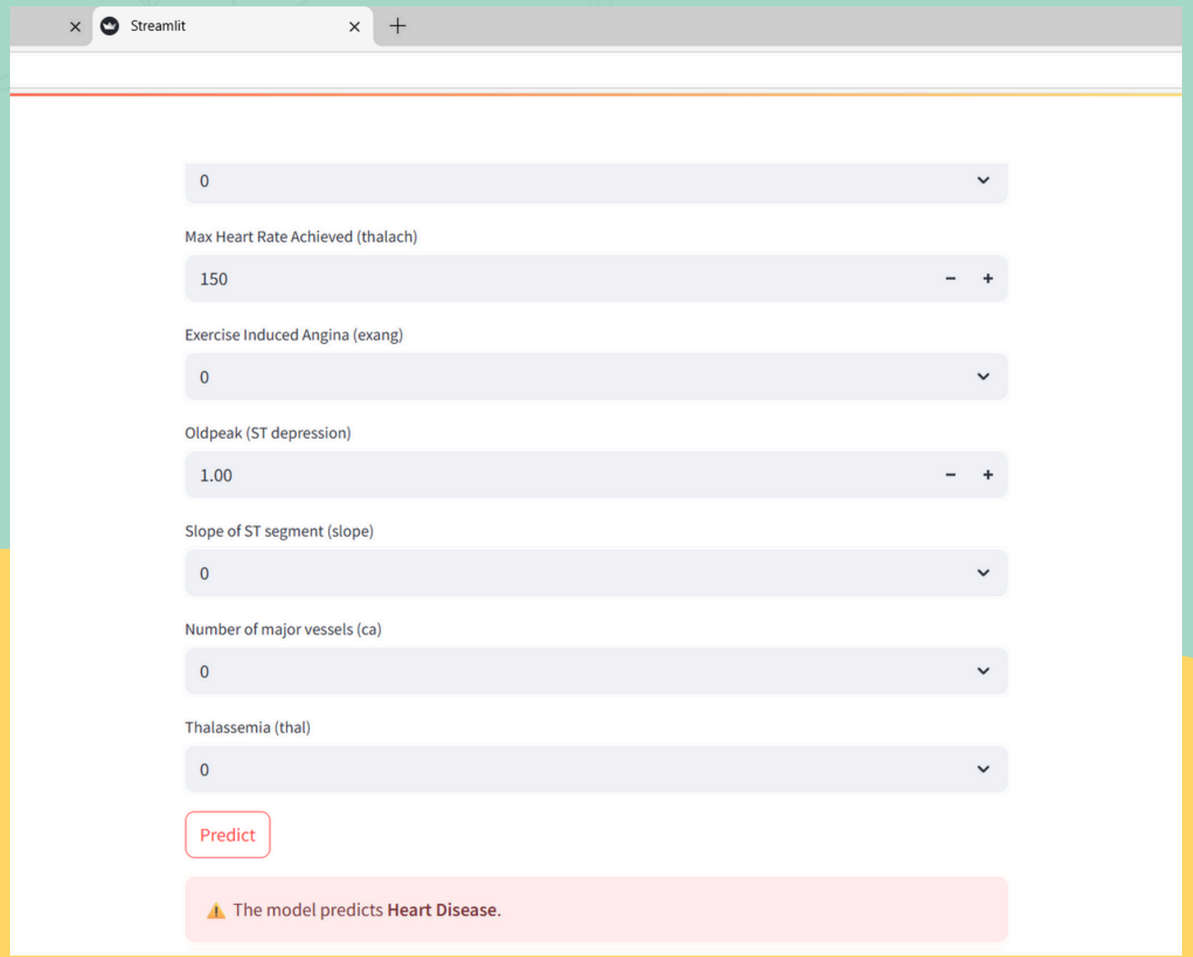




"A code file was developed and executed through the terminal."

OUTPUT





"Upon execution, the Heart Disease Prediction App launches in the browser, presenting a user-friendly interface to input patient health details."

APPLICATION

- Used to predict heart disease risk early using patient health data
- Helps in telemedicine and remote health checkups
- Can be used in health camps, clinics, and mobile apps
- Useful for preventive care in both urban and rural areas
- Supports medical students and researchers in learning ML in healthcare

ADVANTAGES

- Early Detection: Predicts risk before symptoms appear
- Fast & Real-time Results: Thanks to the Python + Streamlit web app
- User-Friendly Interface: Easy for anyone to use without technical skills
- Cost-effective: No expensive tests — just basic health inputs needed
- Scalable: Can be expanded to include more diseases or large datasets
- Data-Driven Decisions: Helps doctors and users make informed health choices

THANK YOU